

DMHelper — Ruleset & Template System

A guide for users, campaign authors, and developers who want to understand, configure, modify, or extend the rules systems shipped with DMHelper.

Table of Contents

1. [Overview](#)
 2. [Core Concepts](#)
 3. [Architecture: Classes and Responsibilities](#)
 4. [How a Ruleset is Loaded](#)
 5. [Modifying the Campaign Ruleset](#)
 6. [Creating a New Ruleset](#)
 7. [Modifying an Existing Ruleset](#)
 8. [Reference: Resource XML Files](#)
 9. [Reference: Resource UI Files](#)
 10. [Property Reference for .ui Files](#)
-

1. Overview

DMHelper is a virtual tabletop (VTT) for Dungeon Masters that supports more than one role-playing system. The differences between systems — how initiative is rolled, how hit points and damage are tracked, what stats a character has, what a monster's stat block looks like, which conditions exist — are isolated behind a small set of pluggable abstractions collectively referred to as the **ruleset and template system**.

A *ruleset* is the combination of:

- A pair of **rule strategies** (RuleInitiative, RuleHealth).
- A set of **data templates** that describe the fields a character or monster has (e.g. character5e.xml).
- A set of **UI templates** that describe how to display and edit those fields (e.g. character5e.ui).
- A list of **conditions** for that system (e.g. conditions-5e.xml).
- A set of optional secondary files: spell data, spell UI, lair-action UI, combatant token UI, default bestiary, etc.

DMHelper currently ships rulesets for D&D 5e (2014), D&D 5e (2024), D&D 2e, and Daggerheart, but the system is designed so that new rulesets can be added without modifying the core application logic.

2. Core Concepts

2.1 Ruleset

A Ruleset is a per-campaign object that wires together the rule strategies and the data/UI files used by that campaign. It is created when a new campaign is started (from a *ruleset template*) and then saved into the campaign file so the campaign always reopens with the same configuration.

2.2 Ruleset Template

A ruleset *template* is the description of an available ruleset, parsed from [resources/ruleset.xml](#) at application startup. Templates appear in the dropdown of the **New Campaign** dialog. A campaign records the *name* of the template it was created from, plus any per-campaign overrides.

2.3 Rule Strategies

Two pluggable strategies determine system-specific combat behaviour:

Strategy	Responsibility
RuleInitiative	How initiative is rolled and how combatants are sorted into turn order.
RuleHealth	How hit points / damage / death work, and whether HP counts up or down.

Each strategy is a small C++ class with a known *type string* (e.g. "5e", "2e", "group", "daggerheart"). The RuleFactory looks up the type string and instantiates the correct subclass.

2.4 Template Object

A TemplateObject is the *data* for a single character, monster, spell, or other rule-driven entity. It exposes a generic key / value interface (e.g. getStringValue("name"), setIntValue("level", 3)) so that the template system can read and write fields without knowing the details of each subclass.

CharacterV2, MonsterClassV2, Spell and similar classes inherit TemplateObject.

2.5 Template Factory

A TemplateFactory is bound to a single data-template .xml file (such as character5e.xml). It knows the schema for that system — which fields exist, what their types are, and what their defaults are — and it loads the matching .ui file to display them. It also performs the binding between widgets and the underlying TemplateObject.

2.6 Template Frame

A `TemplateFrame` is the *editor* — the dialog or widget the user sees when they edit a character or monster. The frame owns the loaded UI widget tree, listens for user edits, and pushes changes back into its `TemplateObject`.

2.7 Field Format

Fields can carry a *display format* (`dmhFormat`) that adds a prefix or suffix, applies digit padding, signs the value, and so on. Fields can also carry a *compute expression* (`dmhCompute`) that derives the value from other fields (e.g. ability modifiers from ability scores). `TemplateFieldFormat` parses both kinds of spec.

2.8 Object Notifier

Because `TemplateObject` cannot itself inherit `QObject` (its concrete subclasses already do), each instance owns a small companion object, `TemplateObjectNotifier`, that emits a `valueChanged(key)` signal when a field is mutated. Computed widgets connect to this notifier to refresh themselves when one of their input fields changes.

2.9 Resource Layout

A `TemplateResourceLayout` is a horizontal layout that displays a "current / maximum" counter — hit points, spell slots, hit dice, bardic inspiration uses, and so on. It is generated at runtime from the schema's `dmh:resource` fields and from the optional `resource.ui` / `spellSlot.ui` widget shells.

3. Architecture: Classes and Responsibilities

Ruleset — [ruleset.h](#)

The per-campaign ruleset container. Holds:

- The active `RuleInitiative` and `RuleHealth` instances.
- Paths to the character data file, character UI file, monster data file, monster UI file, combatant UI file, spell data file, spell UI file, lair-action UI file, default bestiary file, and conditions file.
- The active `Conditions` set and the original `Conditions` set (so condition deltas can be calculated on save).
- Combat-related flags: whether the *Done* checkbox is shown next to each combatant, whether HP counts down, what kind of movement display is in use, and any custom movement ranges.

`Ruleset` is a `CampaignObjectBase`, so it is serialised into the campaign file as part of the normal save flow. On load it asks `RuleFactory` for the named template and uses it to construct the right strategy objects.

RuleFactory — [rulefactory.h](#)

Singleton that owns:

- The list of *ruleset templates* parsed from [resources/ruleset.xml](#) at startup (and from an optional user file).
- The factory methods that map a *type string* to a concrete RuleInitiative or RuleHealth subclass.
- The list of available initiative and health type names (used to populate dropdowns in the campaign-properties UI).

To register a new initiative or health strategy you add a case in RuleFactory::createRuleInitiative() or createRuleHealth() and a matching name in getRuleInitiativeNames() / getRuleHealthNames().

Conditions and condition files

Conditions holds the list of status effects (blinded, grappled, on fire, in stress, etc.) for a campaign. The base list comes from the ruleset template's conditions file (conditions-5e.xml, conditions-daggerheart.xml). Per-campaign additions and removals are stored as a *delta* on save and re-applied on load, so that user-customised conditions survive without duplicating the shipped defaults inside every campaign file.

TemplateFactory — [templatefactory.h](#)

Each TemplateFactory is bound to a single data-template file (for example, the active character template or the active monster template). The factory:

- Parses the .xml template into an internal map of field metadata (name, type, default value, list children).
- Loads the matching .ui file via QUILoader.
- Walks the widget tree and matches widgets to fields using the dmhValue property.
- Reads values from a TemplateObject into widgets, and pushes widget edits back into the object.
- Generates resource widgets at runtime from resource.ui / spellSlot.ui for fields of type dmh:resource.

TemplateObject — [templateobject.h](#)

The data side of the binding. Provides:

- Typed accessors: getStringValue, getIntValue, getBoolValue, getDiceValue, getResourceValue, getListValue, and matching setters.
- List operations: appendListEntry, insertListEntry, removeListEntry, createListEntry.
- Generic XML I/O via readXMLValues / writeXMLValues that uses the associated TemplateFactory to know which fields exist and how to convert them.
- A virtual hook (isAttributeSpecial, getAttributeSpecial, setAttributeSpecial) so subclasses can intercept reads or writes for computed or non-trivial fields.

TemplateFrame — [templateframe.h](#)

A small bridge object owned by an editor dialog or frame. It exposes:

- The .ui file name to load.
- `handleEditBoxChange()`, `handleAddResource()`, `handleRemoveResource()` — entry points called from the templated UI when the user edits a field or adds / removes a resource list entry.
- `postLoadConfiguration()` — called after the UI is loaded so the host can attach buttons, icons, or signal handlers that the .ui file cannot express on its own.
- `localEventFilter()` — for context menus and other event-level work.

Concrete frames are created for each editor: character editor, bestiary (monster) editor, spellbook editor, and the in-battle combatant frame.

TemplateFieldFormat — [templatefieldformat.h](#)

Parses two kinds of strings:

- **Format specs** (`dmhFormat`) such as `int;0..20;signed;prefix:AC or string;suffix: ft..`. The parser produces a `FormatSpec` and helpers `applyFormat`, `applyFormatInt`, `stripFormat`, and `makeValidator`.
- **Compute expressions** (`dmhCompute`) such as `(dexterity - 10) / 2`. The parser produces a `ComputeExpr` AST that can be evaluated against a `TemplateObject` and reports the set of field names it references — used to subscribe the widget to the right change notifications.

TemplateObjectNotifier — [templateobjectnotifier.h](#)

A lightweight `QObject` companion. Each `TemplateObject` lazily constructs one and exposes it via `notifier()`. Computed widgets connect their refresh slot to `valueChanged(QString)` for each field name their compute expression depends on.

This indirection exists because `TemplateObject` itself cannot be a `QObject`: its subclasses (e.g. `CharacterV2`, `MonsterClassV2`) already inherit from `QObject` for their own reasons, and Qt forbids multiple inheritance from `QObject`.

TemplateResourceLayout — [templateresourcelayout.h](#)

A `QHBoxLayout` subclass that draws a current/maximum counter and emits `resourceValueChanged` (or `resourceListValueChanged` for list resources), `addResource`, and `removeResource` signals. The `TemplateFactory` plants one of these layouts wherever the .ui file declares a `dmhWidget="resource"` placeholder.

Rule strategy hierarchies

```
RuleInitiative
├── RuleInitiative5e          type: "5e"
├── RuleInitiative2e          type: "2e"
├── RuleInitiativeGroup        type: "group"
└── RuleInitiativeGroupMonsters type: "groupmonsters"
```

RuleHealth	
└ RuleHealth5e	type: "5e"
└ RuleHealthDaggerheart	type: "daggerheart"

Each subclass defines a small number of methods — how to roll, how to sort, how to apply damage, whether HP counts up or down, and what "dead" / "incapacitated" mean. Adding a new system is largely a matter of subclassing one or both of these and registering the subclass with the RuleFactory.

4. How a Ruleset is Loaded

4.1 At application startup

1. RuleFactory::Initialize() runs and parses [resources/ruleset.xml](#) into a list of ruleset templates. If a user-overrides file exists, it is parsed and merged on top.
2. MonsterFactory, SpellbookFactory, and QuickRef are initialised against the default ruleset, so the bestiary and quick reference are functional even before a campaign is opened.

4.2 When the user creates a new campaign

1. The **New Campaign** dialog calls RuleFactory::getRulesetNames() and shows the result in a dropdown.
2. When the user picks a ruleset, the dialog reads the matching RulesetTemplate and pre-fills the data file, UI file, and rule type choices. The user can override any of these.
3. On OK, a fresh Ruleset is constructed from the template (and any overrides) and attached to the new Campaign. This in turn instantiates the RuleInitiative and RuleHealth objects via RuleFactory.

4.3 When the user opens an existing campaign

1. Ruleset::inputXML() is called with the <ruleset> element from the campaign file. It reads the template name and calls RuleFactory::getRulesetTemplate().
2. Any per-campaign overrides on the <ruleset> element (initiative type, health type, file paths, condition deltas) are applied.
3. The RuleInitiative, RuleHealth, and Conditions objects are constructed.
4. The campaign continues loading, and as characters / monsters are parsed they are bound to the appropriate TemplateFactory so their field values can be read in the right schema.

4.4 When the user opens a character or monster editor

1. The editor instantiates a concrete TemplateFrame (e.g. CharacterTemplateFrame).
2. The frame asks TemplateFactory::loadUITemplate() to load the ruleset's .ui file and gets a populated QWidget tree back.

3. `TemplateFactory::populateWidget()` walks the tree, reads each widget's `dmhValue`, fetches the matching value from the `TemplateObject`, applies any `dmhFormat` decoration, and writes it into the widget.
 4. As the user edits, the frame's `handleEditBoxChange()` (or the resource widget's signals) push values back through the `TemplateObject`. Any computed widgets refresh themselves via the `TemplateObjectNotifier`.
-

5. Modifying the Campaign Ruleset

The campaign ruleset can be edited from the campaign properties (or the campaign tree). Common operations:

5.1 Changing the initiative or health rule

The dropdowns are populated from `RuleFactory::getRuleInitiativeNames()` and `getRuleHealthNames()`. Selecting a different entry constructs a new strategy object and attaches it to the `Ruleset`. The change is recorded as an attribute on the `<ruleset>` element when the campaign is saved.

Note: changing the health rule mid-campaign does not automatically rewrite the existing combatants' hit-point fields. If the new rule uses a different attribute (for example, switching between 5e *count-down* HP and Daggerheart's *threshold* model), existing combatants may need their fields adjusted manually.

5.2 Pointing to a different data or UI file

Each file path on the `Ruleset` (character data, character UI, monster data, monster UI, combatant UI, spell data, spell UI, lair-action UI, bestiary, conditions) can be overridden per-campaign. This is how a campaign can pin itself to a customised character sheet without modifying the global ruleset definition. Overrides are written into the `<ruleset>` element on save.

5.3 Adding or removing conditions

The conditions editor lets the DM add, edit, or hide conditions for the current campaign. The base list comes from the ruleset's conditions file. Only the *delta* — additions and per-condition changes — is saved inside the campaign. This keeps campaign files small and allows shipped conditions to be updated centrally.

5.4 Combatant display flags

The campaign can toggle the *Done* checkbox on combatants and the HP count-up/count-down direction. These are stored on the `Ruleset` and respected by the battle UI.

6. Creating a New Ruleset

A new ruleset is a combination of new C++ rule strategies (only if your system needs initiative or HP behaviour that doesn't already exist), new data templates, new UI templates, and a new entry in `ruleset.xml`. The minimum recipe is:

6.1 Add rule strategy classes (only if needed)

1. Subclass `RuleInitiative` and / or `RuleHealth`. Pick a unique type string for each (e.g. "mysystem").
2. Override the pure virtual methods. For initiative this is `roll`, `sort`, `compare`, and `newRound`. For health this is `applyDamage`, `isDead`, `isIncapacitated`, `healthCountsUp`, and `rollInitial`.
3. Add the new `.cpp` and `.h` files to the explicit source list in `CMakeLists.txt`. The project does not glob — every file must be listed.
4. Register the new types in `RuleFactory`: - Add a branch to `createRuleInitiative()` / `createRuleHealth()`. - Add the type / display-name pair to `getRuleInitiativeNames()` / `getRuleHealthNames()`.

If you can reuse one of the existing strategies (for example, *group* initiative is generic), skip this step and reference the existing type string in `ruleset.xml`.

6.2 Author the data templates

Create the system's data templates under `resources/`:

- `character-mysystem.xml` — character / PC schema.
- `monster-mysystem.xml` — monster / NPC schema.
- `conditions-mysystem.xml` — status effect list.
- Optional: `spell-mysystem.xml` for spell data.

Templates use the `dmh:` element vocabulary:

```
<dmh:template>
  <dmh:attribute type="dmh:string"    name="name"/>
  <dmh:attribute type="dmh:integer"   name="level" default="1"/>
  <dmh:attribute type="dmh:boolean"   name="inspiration"/>
  <dmh:attribute type="dmh:resource"  name="hit-points"/>
  <dmh:attribute type="dmh:dice"      name="hit-dice" default="1d8"/>

  <dmh:element type="dmh:html"       name="notes"/>

  <dmh:element type="dmh:list"       name="spellSlots">
    <dmh:attribute type="dmh:integer" name="level"/>
    <dmh:attribute type="dmh:resource" name="slots"/>
  </dmh:element>
```



```
</dmh:template>
```

Field types:

Type	Stored as	Used for
dmh:string	XML attribute / text	names, freeform short text
dmh:integer	XML attribute	numeric stats
dmh:boolean	XML attribute (0 / 1)	checkboxes
dmh:resource	XML attribute (current,max)	HP, slots, uses
dmh:dice	XML attribute ("1d8+2")	damage, hit dice
dmh:html	XML element (escaped HTML)	rich-text fields
dmh:list	repeated XML elements	lists of structured sub-records

6.3 Author the UI templates in Qt Designer

Open Qt Designer and build the editor UIs:

- resources/ui/character-mysystem.ui
- resources/ui/monster-mysystem.ui
- resources/ui/combatant-mysystem.ui — the in-battle token frame.

For every widget that should be bound to a data field, set the `dmhValue` dynamic property to the field name from your data template. For computed or formatted fields, also set `dmhFormat` and / or `dmhCompute` (see [§10](#)).

The shell of the UI — scroll areas, layouts, tabs, static labels — should always come from the `.ui` file. Programmatic widget creation should be reserved for genuinely runtime-driven content.

6.4 Register the ruleset in `ruleset.xml`

Add an entry to [resources/ruleset.xml](#):

```
<ruleset name="My System"
  initiative="mysystem"
  health="mysystem"
  characterdata="./character-mysystem.xml"
  characterui="./ui/character-mysystem.ui"
  monsterdata="./monster-mysystem.xml"
  monsterui="./ui/monster-mysystem.ui"
  combatantui="./ui/combatant-mysystem.ui"
  conditions="conditions-mysystem.xml"
  hitPointsCountDown="0"
```

```
combatantDone="0"/>
```

The name attribute is what appears in the New Campaign dropdown.

6.5 Update the build

Add every new .cpp/.h to the explicit source list in CMakeLists.txt. Resource .xml and .ui files must also be added to the resource list (typically dmhelper.qrc) so they are bundled with the application.

6.6 Optional: ship a starter bestiary

Place a starter monster collection at resources/DMHelperBestiary-mysystem.xml and reference it from the new ruleset entry's bestiary attribute. This gives users an immediate selection of pre-built creatures.

7. Modifying an Existing Ruleset

Most changes do not require a new ruleset entry. Common modifications:

7.1 Add or rename a field

1. Edit the matching data template (e.g. character5e.xml) to add the new <dmh:attribute> or <dmh:element>.
2. Open the matching .ui file in Qt Designer and add a widget for the new field. Set the widget's dmhValue property to the field name.
3. Existing campaigns will read the new field as the default value declared in the template, and will start saving it once the user edits it.

7.2 Change a computed value

dmhCompute expressions are plain text on the widget. Open the .ui file in Qt Designer and edit the dmhCompute property — no code change is required. The expression can reference any other field in the same TemplateObject.

7.3 Change the display format

Edit the widget's dmhFormat property in Qt Designer. For example, int;0..20;signed;prefix:AC will display the integer with a sign, constrain it to 0..20, and prefix the displayed text with "AC ". The underlying stored value is unchanged.

7.4 Add or remove a condition

Edit the conditions file for the system (e.g. conditions-5e.xml). Each condition has id, title, icon, description, and an optional group for related conditions such as the five 5e exhaustion levels. Existing campaigns will inherit the edited list on next load.

7.5 Tweak combat behaviour

Set or change attributes on the ruleset entry in ruleset.xml:

Attribute	Effect
combatantDone	Show a "done" checkbox per combatant.
hitPointsCountDown	1 for 5e-style countdown, 0 for count-up systems.
movement	distance for a single distance value, or range; short; medium; long for ranged categories.

8. Reference: Resource XML Files

All paths are relative to resources/. "Loaded by" lists the principal consumer; many of these files are also reachable through the Ruleset accessors when a campaign overrides a path.

ruleset.xml

Master registry of available rulesets. Each <ruleset> element describes one playable system: its rule strategy types, the paths to its data and UI templates, its bestiary, its conditions, its spell files, and its combat-display flags.

Loaded by: RuleFactory::Initialize() at application startup. Read indirectly by Ruleset::inputXML() when a campaign is opened, via RuleFactory::getRulesetTemplate().

ruleset.xsd

XML Schema for ruleset.xml. Used as documentation and (where available) for validation while editing.

character5e.xml

Data template for a D&D 5e (2014) player character. Defines the field schema used by the 5e character editor.

Loaded by: TemplateFactory bound to characters when the active ruleset is *DnD 5e 2014*. Pointed to from ruleset.xml via characterdata.

character5e-2024.xml

Data template for a D&D 5e (2024) player character.

Loaded by: the character TemplateFactory when the active ruleset is *DnD 5e 2024*.

character2e.xml

Data template for an AD&D 2nd Edition character.

Loaded by: the character TemplateFactory when the active ruleset is *DnD 2e*.

character-daggerheart.xml

Data template for a Daggerheart character (hope, stress, evasion, domains, ancestries, communities, etc.).

Loaded by: the character TemplateFactory when the active ruleset is *Daggerheart*.

character.xsd

Schema describing the dmh:template vocabulary used by all character* templates. Reference document for authors.

monster5e.xml

Data template for a D&D 5e bestiary entry — armor class, hit points, attributes, actions, reactions, legendary actions, etc.

Loaded by: the monster TemplateFactory when the active ruleset is *DnD 5e 2014* or *DnD 5e 2024* (both reuse the 5e monster schema).

monster2e.xml

Data template for a D&D 2e bestiary entry.

Loaded by: the monster TemplateFactory when the active ruleset is *DnD 2e*.

monster-daggerheart.xml

Data template for a Daggerheart adversary.

Loaded by: the monster TemplateFactory when the active ruleset is *Daggerheart*.

conditions-5e.xml

The 5e status-effect list (blinded, charmed, deafened, exhaustion 1–5, frightened, grappled, incapacitated, invisible, paralysed, petrified, poisoned, prone, restrained, stunned, unconscious).

Loaded by: Ruleset::inputXML() via the ruleset template's conditions attribute.

conditions-daggerheart.xml

The Daggerheart status-effect list.

Loaded by: Ruleset::inputXML() for Daggerheart campaigns.

equipment.xml / equipment.xsd

Reference database of magic items used by the shop / loot generator. Items declare a category, optional attunement requirement, and rarity / probability bands.

Loaded by: the equipment / shop generation subsystem; not tied to any specific ruleset.

shops.xml / shops.xsd

Probability tables describing what shops exist in settlements of each size (hamlet, village, town, city, metropolis) and what kinds of items each shop stocks. Drives random shop generation.

Loaded by: the shop / loot generator.

spell5e.xml

Bundled D&D 5e spell reference data (name, school, level, casting time, range, components, duration, description). Used as the source pool when adding spells to a 5e character.

Loaded by: the spell / spellbook subsystem when a 5e ruleset is active.

spellbook.xml

Schema describing the per-character spellbook structure (prepared, ritual, list of spells).

Loaded by: the spellbook TemplateFactory when the character editor shows the spellbook tab.

spellbook.xsd

XML Schema for spellbook.xml.

calendar.xml / calendar.xsd

Game-world calendar definitions: weeks, months, leap-year rules, and named special days. Drives the in-game date tracker.

Loaded by: the campaign date / calendar subsystem (BasicDate).

quickref_data.xml / quickref_data.xsd

Hierarchical quick-reference content displayed in the in-app reference panel — categories such as conditions, actions, common spells, and short rules summaries.

Loaded by: `QuickRef::Initialize()` at application startup.

tables/

A directory of additional reference tables used by the random-table roller. Each file inside is a self-contained table; new tables can be dropped in without code changes.

dmh_default_frame.png, dmh_default_mask.png

Default token frame and alpha mask used when no custom frame is set on a combatant. Referenced by the combatant rendering path, not by the ruleset schema.

9. Reference: Resource UI Files

All paths are relative to `resources/ui/`. Each `.ui` file is a Qt Designer XML document loaded with `QUiLoader`. Widgets in these files carry the `dmhValue`, `dmhWidget`, `dmhFormat`, `dmhCompute`, and `dmhCondition` dynamic properties described in [§10](#).

character5e.ui

Editor for a D&D 5e (2014) player character. Bound to `character5e.xml`.

Loaded by: the character editor frame when the active ruleset is *DnD 5e 2014*. Path supplied by `Ruleset::getCharacterUIFile()`.

character5e-2024.ui

Editor for a D&D 5e (2024) player character. Bound to `character5e-2024.xml`.

Loaded by: the character editor when the active ruleset is *DnD 5e 2024*.

character2e.ui

Editor for an AD&D 2e player character. Bound to `character2e.xml`.

Loaded by: the character editor when the active ruleset is *DnD 2e*.

character-daggerheart.ui

Editor for a Daggerheart player character. Bound to `character-daggerheart.xml`.

Loaded by: the character editor when the active ruleset is *Daggerheart*.

monster5e.ui

Editor for a D&D 5e bestiary entry. Bound to monster5e.xml.

Loaded by: the bestiary editor when the active ruleset is *DnD 5e 2014*. Path supplied by Ruleset::getMonsterUIFile().

monster5e-2024.ui

Editor for a D&D 5e (2024) bestiary entry. Reuses the 5e monster schema.

Loaded by: the bestiary editor when the active ruleset is *DnD 5e 2024*.

monster2e.ui

Editor for a D&D 2e bestiary entry. Bound to monster2e.xml.

Loaded by: the bestiary editor when the active ruleset is *DnD 2e*.

monster-daggerheart.ui

Editor for a Daggerheart adversary. Bound to monster-daggerheart.xml.

Loaded by: the bestiary editor when the active ruleset is *Daggerheart*.

combatant5e.ui

The 5e in-battle combatant frame — the small pop-out shown next to a combatant's token during an encounter.

Loaded by: the battle frame's combatant template factory when the active ruleset is 5e. Path supplied by Ruleset::getCombatantUIFile().

combatant-daggerheart.ui

The Daggerheart in-battle combatant frame.

Loaded by: the battle frame when the active ruleset is *Daggerheart*.

condition.ui

Widget for a single condition entry — used inside the conditions editor and the condition picker on a combatant.

Loaded by: the conditions editor and combatant condition list.

action.ui

Widget for a single monster action / reaction / legendary action row, used inside the bestiary editor's action lists.

Loaded by: the bestiary editor when populating action lists.

feature.ui

Widget for a single character feature / trait / class feature row.

Loaded by: the character editor when populating feature lists.

experience.ui

Widget for a single experience-point or milestone entry in the character XP log.

Loaded by: the character editor's experience tracker.

resource.ui

Generic shell for a single resource counter (current/maximum). The `TemplateFactory` instantiates this widget once per `dmh:resource` field and wires it up via `TemplateResourceLayout`.

Loaded by: `TemplateFactory::createResourceWidget()` whenever a resource field is encountered.

spellSlot.ui

Specialised resource shell for spell slots, with slot-level decoration not present in the generic `resource.ui`.

Loaded by: `TemplateFactory::createResourceWidget()` for fields whose widget hint is `spellSlot`.

spellbook5e.ui

Editor for the 5e spellbook (the per-character list of known and prepared spells). Bound to `spellbook.xml`.

Loaded by: the spellbook editor when the active ruleset has a spell UI configured.

10. Property Reference for .ui Files

All bindings are expressed as Qt Designer *dynamic properties* on the relevant widget. Add them in Qt Designer's property editor (or, if necessary, through the *Add Dynamic Property* dialog).

dmhValue (*string*)

The data field name in the data template. Required for every widget that should be bound. Example: name, armor_class, hit-points.

For widgets inside a `dmh:list`, use the field name relative to the list entry; the factory resolves the list context automatically based on widget parentage.

dmhWidget (string)

A widget *kind* hint. Used when the factory needs to do something beyond reading or writing a plain value. The most common value is `resource`, which causes the factory to replace the placeholder with a `TemplateResourceLayout`. `spellSlot` selects the spell-slot variant of the resource shell.

dmhFormat (string)

A semicolon-separated format spec parsed by `TemplateFieldFormat`. Components include:

- `int` — treat the value as an integer.
- `string` — treat the value as a string.
- `min..max` — clamp / validate to a numeric range.
- `signed` — always show a leading + for positive integers.
- `pad:N` — left-pad an integer to N digits.
- `prefix:TEXT` — prepend TEXT to the displayed value.
- `suffix:TEXT` — append TEXT to the displayed value.

The format is applied when displaying and stripped before writing back to the model, so the user sees AC 15 while the underlying value remains 15.

dmhCompute (string)

A simple arithmetic expression that derives the widget's value from other fields on the same `TemplateObject`. Identifiers in the expression are field names. Computed widgets are read-only and refresh themselves whenever any of their referenced fields changes.

Example: `(dexterity - 10) / 2` for a 5e ability modifier.

dmhCondition (string)

An optional visibility condition. The widget is shown when the condition evaluates true and hidden when it evaluates false. This allows a single `.ui` file to host fields that only apply to a subset of subclasses or campaign settings.

Appendix A: Files You Should *Not* Hand-Edit

- `*.ui` — always edit in Qt Designer; never edit the XML directly.

- *.qrc — only modify with care; do not restructure paths.
- Pre-built binary directories (vlc32/, vlc64/, vlcMac/, bin-win*/, bin-macos/).

Appendix B: Quick Glossary

Term	Meaning
Ruleset	The per-campaign bundle of rule strategies, templates, and files.
Ruleset template	A <ruleset> entry in ruleset.xml; the recipe for new campaigns.
Rule strategy	RuleInitiative or RuleHealth subclass.
Data template	dmh:template .xml file that defines a schema.
UI template	A Qt Designer .ui file with dmh* dynamic properties.
Template factory	The C++ binder between a data template, a UI template, and a TemplateObject.
Template frame	The editor (dialog or widget) hosting a templated UI.
Template object	A data row instance — a character, monster, or spell.
Resource	A current/maximum paired field, typically with +/- controls.
Condition	A status effect such as <i>blinded</i> , <i>grappled</i> , or <i>in stress</i> .
Condition delta	The per-campaign difference from the ruleset's default conditions.